VideoMASys

Production-Oriented Multi-Task Video Understanding and Editing System

System Design, Version 0.1

Architecture proposal based on Structured Video Programming principles

5 September 2026

Abstract

VideoMASys is a multi-task video intelligence platform for question answering, referring-object segmentation, object tracking, and non-destructive video editing. Its central design is to process a video once, cache a hierarchical semantic representation, and inspect only the time ranges and pixels required for each request. A language model supplies task decomposition and semantic reasoning through a typed plan. Deterministic validators, schedulers, tools, and renderers execute that plan. This preserves the intelligence of agentic systems without placing an unbounded chain of model calls or arbitrary generated code on the production request path.

Contents

1 Executive Summary **3**
1.1 Core decisions . . . 3

2 Goals and Non-Goals **3**
2.1 Goals . . . 3
2.2 Non-goals for the first release . . . 4

3 System Context and Architecture **4**
3.1 Relationship to Structured Video Programming . . . 4
3.2 Prototype model-serving strategy . . . 5

4 Hierarchical Video Cache **5**
4.1 Canonical node model . . . 6
4.2 Storage responsibilities . . . 6
4.3 Prototype database choice . . . 6

5 Agents, Tools, and Responsibilities **7**
5.1 Agent contract . . . 7
5.2 Core roles . . . 7

6 Typed Task Plan **8**
6.1 Example plan . . . 8
6.2 Initial operator set . . . 8

7 Common Evidence Pipeline **9**
7.1 Retrieval strategy . . . 9

8 Task Workflows **10**
8.1 Video question answering . . . 10
8.2 Referring-object segmentation . . . 10
8.3 Object tracking . . . 10
8.4 Video editing . . . 10

9 Caching and Reuse **10**
9.1 Cache levels . . . 10
9.2 Invalidation . . . 11

10 Latency and Efficiency **11**
10.1 Service classes . . . 11
10.2 Primary optimizations . . . 11

11 APIs and Job Lifecycle **12**
11.1 External API . . . 12
11.2 Unified task request . . . 12
11.3 Lifecycle . . . 12

12 Deployment and Scaling **12**
12.1 Initial topology . . . 12
12.2 Scheduling . . . 13

13 Reliability, Security, and Safety **13**

14 Observability and Evaluation **13**

15 Implementation Roadmap **14**
15.1 Phase 1: Foundation and vertical slice . . . 14
15.2 Phase 2: Object intelligence . . . 14
15.3 Phase 3: Editing . . . 14
15.4 Phase 4: Production hardening . . . 14

16 Initial Acceptance Criteria **15**

17 Open Decisions **15**

1 Executive Summary

The system is divided into two paths:

1. **Offline ingestion** converts source media into reusable temporal, textual, visual, and object artifacts. It creates video, scene, shot, event, transcript, OCR, and keyframe records and indexes their embeddings.
2. **Online task execution** compiles a user request into a bounded, typed workflow; retrieves the most relevant cached nodes; and escalates to pixel-level analysis only when cached evidence is insufficient.

The architecture deliberately distinguishes an **agent** from an LLM call. An agent is a role with a typed input, typed output, policy, budget, and trace. Some agents use an LLM, while retrieval, temporal reasoning, media decoding, mask propagation, tracking, rendering, and most validation remain deterministic services. In the normal case, a request uses one planning call and at most one batched visual-model call.

1.1 Core decisions

- Use an LLM as a semantic planner and bounded replanner.
- Generate a typed intermediate representation (IR), not Python or shell.
- Use a deterministic compiler, validator, and workflow executor.
- Separate reusable video understanding from task-specific execution.
- Treat the vector database as a semantic index, not as media storage.
- Use PostgreSQL with pgvector for both authoritative metadata and the prototype vector index; preserve a `VectorIndex` interface for later scale-out.
- Keep originals and derived media in object storage; keep durable state in a relational database; keep embeddings in a replaceable vector index.
- Start as a modular service plus dedicated CPU/GPU workers. Split into additional services only when measured scaling requirements justify it.
- Make every result reproducible through evidence identifiers, timestamps, model versions, plan versions, and an execution trace.

2 Goals and Non-Goals

2.1 Goals

- Support multiple tasks through one video representation and tool layer.
- Avoid reading or decoding the complete original video at query time.
- Provide bounded latency through progressive evidence acquisition.
- Support synchronous fast paths and asynchronous long-running jobs.
- Allow models, vector stores, and media processors to be replaced behind stable interfaces.
- Produce inspectable evidence, provenance, confidence, and task artifacts.
- Enforce tenant isolation, media access controls, quotas, and safe execution.

2.2 Non-goals for the first release

- Exhaustively detect and track every object in every uploaded video.
- Provide frame-perfect professional editing for arbitrary natural-language instructions without review.
- Permit agents to execute arbitrary source code or invoke unregistered tools.
- Guarantee a single model supports every language, domain, and media format.
- Begin with a large collection of independently deployed microservices.

3 System Context and Architecture

Figure 1 shows the common ingestion, planning, evidence, task, and verification layers. Task branches share the same cached hierarchy and targeted media-access mechanisms.

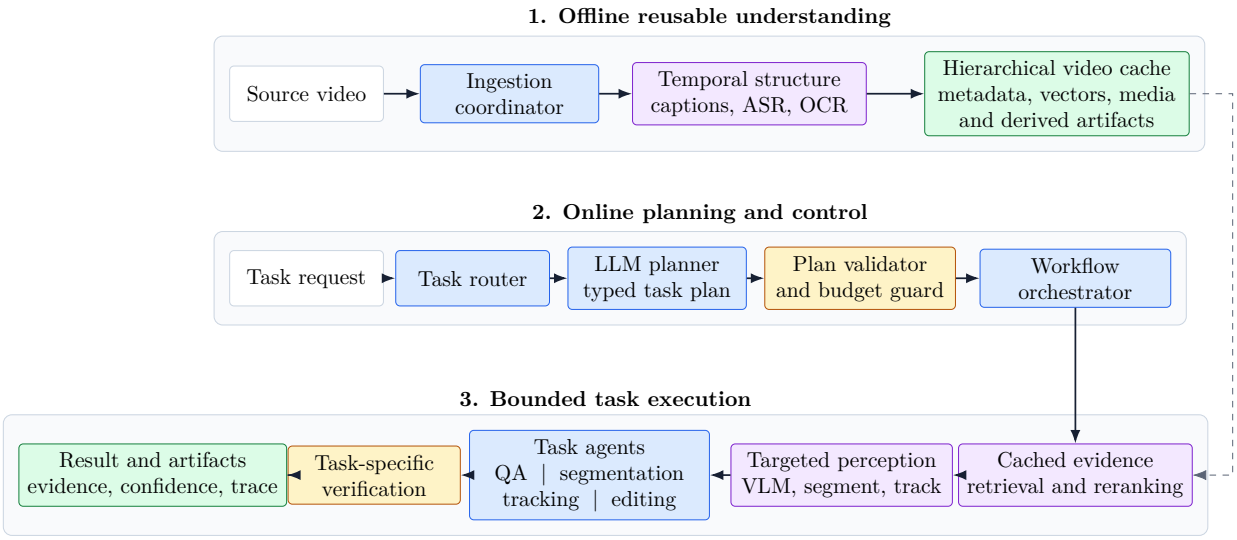


Figure 1: General-purpose architecture. Each swimlane has one flow direction; the only cross-lane data path is the dashed cache-access route. Blue nodes are agent roles, purple nodes are workers, and amber nodes enforce policy.

3.1 Relationship to Structured Video Programming

The design retains four useful ideas from StructuredVP: hierarchical cached video structure, coarse-to-fine evidence selection, typed locate/relate operations, and explicit program traces. It generalizes those ideas beyond QA and segmentation and changes the execution model for production use:

- A task plan is declarative IR rather than generated Python.
- Tool calls are registered, typed, permissioned, metered, and cancellable.
- Plans and results are durable jobs rather than state held only in one WebSocket connection.
- Object nodes may be materialized lazily and reused by later tasks.
- Editing produces a validated edit decision list before any rendering.

3.2 Prototype model-serving strategy

The local prototype uses one instruction-tuned **Qwen3-VL** checkpoint as the shared reasoning and visual-language model. It is hosted by vLLM and exposed through an OpenAI-compatible `/v1/chat/completions` endpoint. A VideoMASys `ModelGateway` is the only component allowed to call that endpoint; agents depend on the gateway interface rather than on vLLM, Qwen-specific request objects, or a hard-coded base URL. Qwen3-VL is used instead of the text-only Qwen3 family because ingestion and evidence roles must accept images or sampled video.

All model-backed roles share the server but retain separate system prompts, Pydantic response schemas, token limits, temperatures, deadlines, priorities, and trace labels. The gateway requests JSON-schema-constrained output for plans, scene/event descriptions, evidence observations, edit decisions, and verifier decisions. This keeps role behavior independently testable while allowing vLLM to batch requests against one resident model.

Role	Model input	Structured output
Scene/event analyst	Candidate shot boundaries plus uniformly sampled proxy frames	Semantic scene/event ranges, labels, boundary confidence
Captioner	Frames or a short proxy clip from one bounded node	Caption, entities, actions, salient text, confidence
Planner/replanner	User intent, task schema, cache summary, budgets	Typed task plan or one bounded plan revision
Visual analyst	Question/purpose plus retrieved frames or clips	Evidence observations tied to timestamps and input artifact IDs
Task agents	Validated evidence bundle and requested output contract	Answer, object prompt, tracking intent, or edit decision list
Verifier	Plan, evidence, output, and task-specific checks	Accept, partial, reject, or bounded evidence-expansion request

Table 1: Prototype roles served by one Qwen3-VL endpoint.

Media probing, decoding, and initial shot-change detection remain deterministic. They generate bounded candidate windows for the scene/event analyst, so the VLM does not ingest an entire long source video in one request. Pixel masks, temporal mask propagation, identity tracking, embeddings, and rendering remain registered specialist tools: a general VLM may select and parameterize these tools but does not replace their pixel-accurate outputs.

This single-model arrangement is a prototype optimization, not a permanent coupling. The gateway records latency, tokens, visual inputs, failures, and quality by role. A later deployment can route any role to a different local or hosted API without changing plans, agent schemas, or orchestration code.

4 Hierarchical Video Cache

The cache is a graph with a dominant temporal hierarchy rather than a single flat embedding collection. Nodes carry stable identifiers and can reference derived artifacts without embedding large binary payloads in the database.

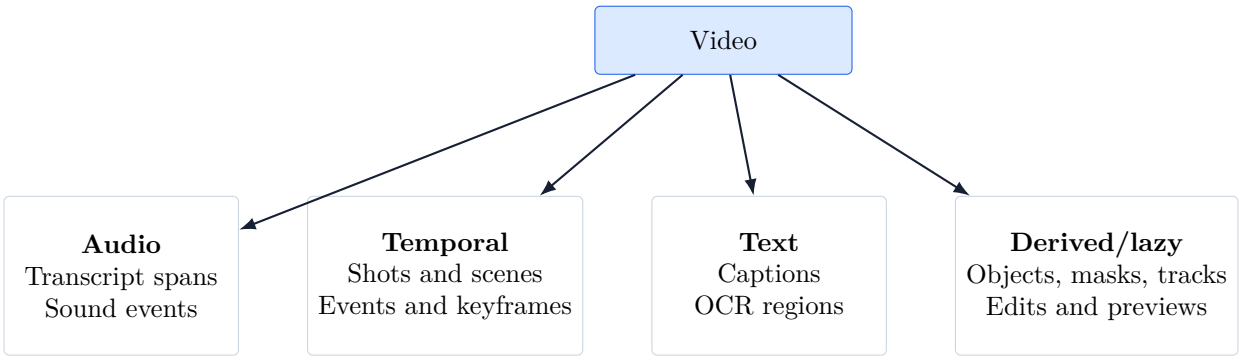


Figure 2: Logical cached representation. Objects, masks, tracks, and editing artifacts are created lazily unless a deployment explicitly enables eager processing for a domain.

4.1 Canonical node model

Every searchable node should contain:

- `id`, `tenant_id`, `video_id`, `parent_id`, and `node_type`;
- half-open time range `[start_ms, end_ms)` and optional frame range;
- text description, transcript text, OCR text, and structured attributes;
- pointers to keyframes, proxy clips, masks, tracks, or render outputs;
- embedding references and lexical-index fields;
- model, prompt, preprocessing, and schema versions;
- content hash, creation time, quality score, and provenance.

Half-open millisecond intervals are canonical. Frame numbers are auxiliary because variable-frame-rate media and transcoded proxies do not preserve one universal frame index.

4.2 Storage responsibilities

Store	Owns	Does not own
Object storage	Originals, proxy clips, keyframes, audio, masks, overlays, rendered edits	Workflow state or searchable metadata
Relational DB	Tenants, videos, temporal nodes, task jobs, plans, leases, lineage, permissions	Large media blobs
Vector index	Embeddings and filterable identifiers for semantic retrieval	Authoritative metadata or media
Artifact cache	Versioned VLM observations, decoded ranges, mask propagation, tracks, retrieval and answer results	The only durable copy of results

Table 2: Storage boundaries. The vector index is a derived, rebuildable index.

4.3 Prototype database choice

The prototype uses PostgreSQL with pgvector. Video hierarchy, temporal ranges, jobs, plans, permissions, provenance, and embedding rows remain transactionally consistent in one service.

B-tree indexes cover tenant/video/type filters, GiST indexes cover temporal overlap, GIN indexes cover JSON and lexical fields, and HNSW indexes cover text and visual embeddings. The application accesses vector search through a `VectorIndex` port rather than issuing pgvector-specific queries outside the storage adapter.

Milvus is a later scale-out option, not an authoritative store. If benchmarks show that vector retrieval misses its latency objective at the required vector count or query concurrency, an outbox-driven indexer can project immutable node IDs and embeddings from PostgreSQL into Milvus. Search results are then hydrated and authorized from PostgreSQL. This preserves one source of truth and makes the external vector index rebuildable.

5 Agents, Tools, and Responsibilities

5.1 Agent contract

Each agent has a narrow contract:

- versioned input and output schemas;
- allowed tools and data scopes;
- latency, token, frame, duration, and monetary budgets;
- retry and fallback policy;
- deterministic correlation identifiers and trace events;
- explicit confidence and failure categories.

Agents do not communicate through unbounded conversational transcripts. They exchange state through a task context and immutable, typed evidence records.

5.2 Core roles

Role	Implementation	Responsibility
Ingestion coordinator	Deterministic workflow	Schedules probing, transcoding, temporal structuring, descriptions, embeddings, and indexing.
Task router	Rules or small model	Classifies task type, synchrony, and likely cost; direct-routes simple operations.
Planner	LLM with structured output	Decomposes complex intent into typed, bounded operations and declares required output.
Plan validator/compiler	Deterministic	Type-checks the graph, validates permissions and budgets, resolves tool versions, and creates executable jobs.
Retrieval agent	Deterministic search/rerank	Searches captions, transcripts, OCR, temporal nodes, and cached object artifacts.
Visual analyst	VLM worker	Examines only selected keyframes or proxy clips and emits observations with temporal provenance.
Object specialist	Vision workers	Detects, segments, propagates, tracks, and re-identifies objects within bounded ranges.
QA agent	LLM or deterministic function	Answers from an evidence bundle; handles counting and comparison deterministically where possible.
Editing agent	LLM plus deterministic compiler	Produces a typed edit decision list; never emits executable shell commands.
Verifier	Rules, metrics, optional model	Checks evidence sufficiency and task-specific output validity; may request one bounded expansion.

Table 3: Agent roles. An agent is an operational boundary, not necessarily a separate language-model invocation.

6 Typed Task Plan

The planner returns a directed acyclic graph expressed as versioned JSON. The plan separates **evidence operations**, which discover or materialize facts, from **task operations**, which produce an answer or artifact.

6.1 Example plan

Listing 1: Illustrative plan for a tracking request.

```
{
  "schema_version": "1.0",
  "task": "object_tracking",
  "video_ids": ["vid_01"],
  "steps": [
    {
      "id": "find_seed",
      "operator": "search_events",
      "inputs": [],
      "arguments": {"query": "red car enters the junction", "limit": 3}
    },
    {
      "id": "inspect_seed",
      "operator": "inspect_visual",
      "inputs": ["find_seed"],
      "arguments": {"purpose": "localize the red car"}
    },
    {
      "id": "track",
      "operator": "track_object",
      "inputs": ["inspect_seed"],
      "arguments": {"direction": "both", "max_duration_ms": 90000}
    },
    {
      "id": "verify",
      "operator": "verify_track",
      "inputs": ["track"],
      "arguments": {"minimum_continuity": 0.85}
    }
  ],
  "output": {"from": "verify", "format": "track_json_and_overlay"},
  "budget": {"deadline_ms": 30000, "max_visual_seconds": 90,
             "max_replans": 1}
}
```

6.2 Initial operator set

- Retrieval: `search_videos`, `search_scenes`, `search_events`, `search_transcript`, `search_ocr`.
- Temporal: `before`, `after`, `around`, `overlap`, `within`, `expand_range`, `merge_ranges`.
- Media: `fetch_keyframes`, `decode_range`, `sample_frames`, `inspect_visual`.
- Objects: `detect_object`, `segment_object`, `propagate_mask`, `track_object`, and `reidentify_object`.
- QA: `count`, `compare`, `aggregate_evidence`, `compose_answer`, `verify_answer`.
- Editing: `select_ranges`, `trim`, `concat`, `crop`, `overlay`, `replace_audio`, `compile_timeline`, `render`, `verify_render`.

Every operator declares accepted input types, output type, side effects, cost class, cache behavior, and whether it may run synchronously. Unknown operators, cycles, invalid references, and over-budget plans are rejected before execution.

7 Common Evidence Pipeline

The orchestrator first attempts to satisfy a task from cached textual and visual evidence. It progressively acquires more expensive evidence only when the plan or verifier requires it.

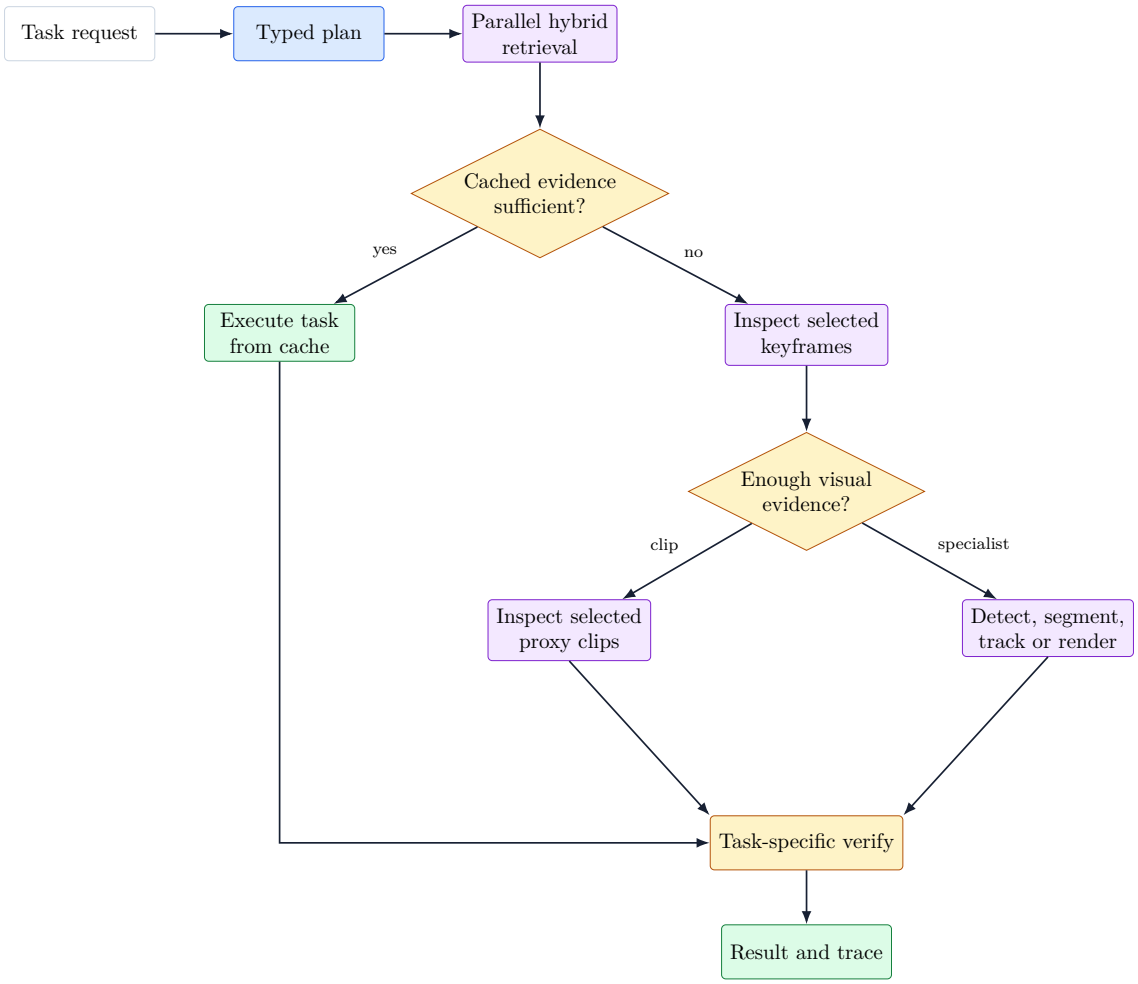


Figure 3: Progressive evidence acquisition. The common path stays cheap; deep perception and rendering are explicit escalation paths. The verifier may request one bounded evidence expansion through the orchestrator.

7.1 Retrieval strategy

1. Apply tenant, video, time, language, and node-type filters.
2. Search dense embeddings and lexical fields in parallel.
3. Fuse and deduplicate results, preserving parent/child relationships.
4. Rerank a small candidate set using query–caption relevance and temporal constraints.
5. Add bounded neighboring context around strong matches.
6. Return an evidence bundle with scores, timestamps, descriptions, and artifact pointers—not full video bytes.

Initial defaults should cap retrieval at 20 candidates, visual inspection at three ranges, combined decoded duration at 60 seconds, sampled frames at 32, and replanning at one attempt. These are deployment defaults rather than values embedded in the plan schema.

8 Task Workflows

8.1 Video question answering

The planner identifies whether the question is factual, temporal, causal, counting, comparative, or object-centric. Transcript, OCR, captions, and event nodes are retrieved first. A visual analyst receives a single batch containing the selected evidence. Counting, sorting, and simple comparisons are performed by deterministic operators when their inputs are reliable. The result contains an answer, confidence, supporting node IDs, timestamps, and limitations.

8.2 Referring-object segmentation

The system retrieves likely temporal windows before invoking segmentation. A visual grounding step chooses one or more seed frames and object prompts. A segmentation worker generates seed masks, and a propagation worker extends them through the requested interval. The verifier checks mask continuity, area jumps, boundary confidence, and prompt consistency. Outputs may include run-length encoded masks, alpha video, overlays, and quality metrics.

8.3 Object tracking

Tracking begins from a grounded seed rather than scanning the full video by default. Propagation proceeds within a bounded interval, with re-identification at scene cuts or after loss. Long videos are divided into jobs that can run in parallel and merge through identity constraints. Outputs include timestamped boxes or masks, confidence, visibility state, and an optional overlay video.

8.4 Video editing

Editing is non-destructive. The editing agent converts natural-language intent into a typed edit decision list (EDL) referencing source assets and half-open time ranges. The compiler validates bounds, codecs, transitions, output aspect ratio, and overlay assets. A deterministic renderer produces a preview and then the final artifact. The system retains the EDL, render manifest, source hashes, and media-tool version so an edit can be reproduced.

Examples include semantic highlight extraction, removal of matching scenes, object-focused crops, reframing around a track, subtitle insertion, and audio replacement. Destructive overwrite of source media is prohibited.

9 Caching and Reuse

9.1 Cache levels

1. **Ingestion cache**: probes, scene boundaries, transcripts, OCR, captions, keyframes, and embeddings.
2. **Evidence cache**: query retrieval results and bounded VLM observations for a video range.
3. **Object cache**: detections, seed masks, propagated masks, tracklets, and re-identification embeddings.
4. **Task cache**: validated answers, segmentation exports, track exports, EDLs, previews, and final renders.

A cache key must include the tenant scope, content hash, exact time range, input parameters, model identifier, model revision, prompt revision, preprocessing revision, and output schema version. Cache writes should use single-flight deduplication so concurrent identical requests do not repeat GPU work.

9.2 Invalidation

Artifacts are immutable. A new model or preprocessing version writes a new artifact rather than mutating the old one. Database pointers identify the active version. Garbage collection deletes unreferenced derived artifacts only after a retention period; source-media deletion follows tenant policy and is separately audited.

10 Latency and Efficiency

10.1 Service classes

Class	Initial objective	Typical use
Interactive cache	p95 below 2 s	Metadata, transcript, OCR, or sufficiently descriptive cached nodes
Interactive visual	p95 below 8 s	QA or localization using one batched set of keyframes/proxy clips
Deep perception	Job accepted below 1 s; completion target 10–60 s	Segmentation and bounded tracking
Render	Job accepted below 1 s; duration-dependent completion	Editing, overlay generation, transcoding, and long-video tracking

Table 4: Latency objectives are starting SLOs and must be calibrated against chosen models, media duration, output resolution, and hardware.

10.2 Primary optimizations

- Keep models resident in dedicated workers; never load them per request.
- Batch all selected frames or clips for a task into one visual call.
- Decode low-resolution proxy media first and originals only when output quality requires them.
- Run independent lexical, vector, metadata, transcript, and OCR searches concurrently.
- Use dynamic batching and length-aware queues for GPU inference.
- Route simple plans to rules or small models and complex plans to a stronger planner.
- Apply backpressure at admission and queue boundaries rather than allowing unbounded GPU queues.
- Propagate deadlines and cancellation tokens through every operation.
- Use semantic result caches carefully; never cross tenant or permission scopes.

11 APIs and Job Lifecycle

11.1 External API

An initial HTTP API should expose:

- `POST /v1/videos`: register an object URI or begin an upload;
- `GET /v1/videos/{id}`: metadata and ingestion readiness;
- `POST /v1/tasks`: submit any supported task with constraints;
- `GET /v1/tasks/{id}`: status, progress, result, and errors;
- `GET /v1/tasks/{id}/events`: server-sent progress events;
- `POST /v1/tasks/{id}/cancel`: cooperative cancellation;
- `GET /v1/artifacts/{id}`: authorized artifact metadata and a short-lived download URL.

11.2 Unified task request

```
{
  "task_type": "qa | segmentation | tracking | editing",
  "video_ids": ["vid_01"],
  "instruction": "Track the cyclist and blur their face",
  "constraints": {
    "time_range_ms": [0, 120000],
    "latency_class": "balanced",
    "maximum_visual_seconds": 60
  },
  "output": {
    "formats": ["track_json", "edited_mp4"],
    "include_trace": true
  },
  "idempotency_key": "client-generated-key"
}
```

11.3 Lifecycle

Jobs transition through `accepted`, `planning`, `retrieving`, `executing`, `verifying`, and a terminal state of `succeeded`, `partially_succeeded`, `failed`, or `cancelled`. Step attempts, leases, and outputs are durable. Workers must be idempotent because a lease can expire and a step can be retried.

12 Deployment and Scaling

12.1 Initial topology

The first production-like deployment should be intentionally small:

- one stateless API/orchestrator service;
- PostgreSQL for metadata, jobs, plans, and leases;
- a vector index behind a repository interface;
- S3-compatible object storage;

- a durable task queue;
- CPU media workers for probing, slicing, and rendering;
- one or more persistent GPU model workers;
- OpenTelemetry-compatible traces, metrics, and structured logs.

This can be deployed locally with containers and in production with the same logical components. Separate planner, retrieval, object, and rendering services only when they require independent scaling, fault isolation, or ownership.

12.2 Scheduling

Queues should be split by resource class: short CPU, long CPU/render, small GPU, large GPU, and latency-sensitive GPU. Admission control estimates cost from video duration, selected ranges, resolution, model, and output format. Tenant quotas constrain concurrent tasks, stored bytes, GPU seconds, and planner tokens.

13 Reliability, Security, and Safety

- Authenticate every API call and authorize every video and artifact.
- Use scoped object-store URLs with short expiry; do not give model workers general storage credentials.
- Validate MIME type and media structure, enforce byte/duration limits, and isolate media parsers and renderers.
- Execute only registered operators. Generated Python, imports, shell commands, arbitrary URLs, and arbitrary filesystem paths are not plan features.
- Keep source media immutable and require a new artifact ID for every edit.
- Apply per-step timeouts, memory limits, cancellation, retries with jitter, and circuit breakers around external model providers.
- Redact secrets and signed URLs from logs and model prompts.
- Record who requested an edit, which inputs were used, and which artifact was produced.
- Make partial success explicit, especially for occluded tracks or low-confidence masks.

14 Observability and Evaluation

Each task creates a trace rooted at `task_id`. Spans include planning, validation, each retrieval source, reranking, media fetch/decode, model queue and inference time, cache access, task execution, verification, and artifact upload.

Core operational metrics include:

- end-to-end p50/p95/p99 latency by task and service class;
- queue wait, model inference, decoded seconds, sampled frames, and GPU seconds per task;
- cache hit rate at each cache level;
- plan validation failure and replan rate;

- evidence expansion rate and retrieved-duration ratio;
- task success, partial success, cancellation, and failure categories;
- cost per successful task and artifact storage growth.

Quality evaluation is task-specific:

- QA: exact/fuzzy accuracy, evidence recall, timestamp precision, and calibration;
- segmentation: region and boundary metrics, temporal consistency, and identity-switch rate;
- tracking: HOTA/ID-style metrics, track continuity, and recovery after occlusion or scene cuts;
- editing: instruction satisfaction, edit-boundary accuracy, render validity, preservation of unedited content, and human preference.

Every benchmark should also report latency, GPU seconds, planner tokens, decoded video duration, and cache state. Accuracy without cost and latency is not an adequate measure of this system.

15 Implementation Roadmap

15.1 Phase 1: Foundation and vertical slice

- Establish package boundaries, configuration, schemas, and local services.
- Register video, ingest scenes/keyframes/transcript/captions, and index nodes.
- Implement typed plans, validator, tool registry, job state, and traces.
- Deliver QA from cached evidence plus targeted keyframes.

15.2 Phase 2: Object intelligence

- Add object grounding, seed masks, mask propagation, and track storage.
- Deliver referring segmentation and bounded tracking.
- Add object-artifact reuse and task-specific verification.

15.3 Phase 3: Editing

- Define the edit decision list and deterministic compiler.
- Add preview rendering, artifact lineage, and render validation.
- Support semantic cuts, concatenation, reframing from tracks, blur, subtitles, overlays, and audio operations.

15.4 Phase 4: Production hardening

- Add authentication, tenant quotas, retention policy, load tests, disaster recovery, autoscaling, and deployment automation.
- Calibrate routing and confidence thresholds from benchmark and production traces.
- Introduce additional service separation only where measurements justify it.

16 Initial Acceptance Criteria

The first integrated release is acceptable when it can:

1. ingest a video asynchronously and expose a versioned hierarchy;
2. answer a question using no more than three retrieved ranges and return timestamped evidence;
3. segment and track a referred object over a bounded interval and cache the resulting masks or tracks;
4. compile an editing instruction to an inspectable EDL and render a new, non-destructive artifact;
5. reject invalid or over-budget plans before execution;
6. resume or safely retry work after a worker failure;
7. show a trace that connects the request, plan, evidence, model/tool versions, task output, and final artifact;
8. enforce that one tenant cannot retrieve another tenant's media, vectors, metadata, or cached artifacts.

17 Open Decisions

The implementation phase should resolve these choices through small benchmarks:

- the measured vector-count, concurrency, recall, and latency threshold at which the `VectorIndex` implementation should move from pgvector to Milvus;
- the Qwen3-VL checkpoint size, quantization, context limit, and vLLM concurrency settings for the available prototype hardware;
- the first replacement targets and routing policy behind the model gateway;
- mask propagation, tracking, and re-identification model families;
- proxy resolutions and frame-sampling policy by content domain;
- synchronous limits and default budgets per task;
- media renderer and the exact versioned EDL schema.

These are replaceable implementation decisions. The stable architectural contract is the typed plan, shared evidence model, hierarchical cache, bounded tool execution, immutable artifacts, and full provenance.